



空间数据库原理与应用

第四讲：数据库标准SQL语言（二）

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

05

SQL复杂查询

06

视图

07

SQL查询的实现过程



表名: Student 说明: 学生基本信息表

属性列	数据类型	长度	空值	列约束	说明
Sno	char()	8	Not Null	PK	学生学号
Sname	char()	10	Not Null		学生姓名
Ssex	char()	2	Null		学生性别
Sage	integer		Null		学生年龄
Dno	char()	2	Null		学生所属系别
Sclass	char()	6	Null		学生班级

表名: Course 说明: 课程信息表

属性列	数据类型	长度	空值	列约束	说明
Cno	char()	3	Not Null	PK	课程编号
Cname	char()	12	Not Null		课程名称
Tno	integer		Null		教师编号
Chours	Float(1)		Null		课程学时
Credit	char()	3	Null		课程学分

表名: SC 说明: 选课表

属性列	数据类型	长度	空值	列约束	说明
Sno	char()	8	Not Null	FK	学生学号
Cno	char()	3	Not Null	FK	课程编号
Score	float(1)		Null		课程成绩

表名: dept 说明: 院系信息表

属性列	数据类型	长度	空值	列约束	说明
Dno	char()	2	Not Null		系编号
Dname	char()	10	Not Null		系名称
Dean	char()	10	Null		系主任

表名: Teacher 说明: 教师信息表

属性列	数据类型	长度	空值	列约束	说明
Tno	char()	3	Not Null	FK	教师编号
Tname	char()	10	Not Null		教师姓名
Dno	char()	2	Null		所属系号
Salary	float(2)		Null		教师工资

创建选课表

Create Table SC (Sno char(8), Cno char(3),Score float(1) , Primary key (Sno, Cno));

为 “dept”表添加 “Count”列（数据类型为char，长度为3，允许为空）

ALTER TABLE dept ADD Count char(3) NULL;

向student表插入一条记录，学号201001，姓名为王小五，性别为男，年龄20，系号为11，其余字段为NULL

INSERT INTO student(Sno, Sname, Ssex, Sage, Dno) VALUES (201001, '王小五', '男', 20, '11');

修改course表记录，将所有记录的学分改为4，学时改为64

UPDATE course SET Chours=64, Credit=4;

删除course表记录，将课程号为123的记录删除

DELETE FROM course WHERE Cno='123';



SQL语句书写

表名: Student		说明: 学生基本信息表			
属性列	数据类型	长度	空值	列约束	说明
Sno	char()	8	Not Null	PK	学生学号
Sname	char()	10	Not Null		学生姓名
Ssex	char()	2	Null		学生性别
Sage	integer		Null		学生年龄
Dno	char()	2	Null		学生所属系别
Sclass	char()	6	Null		学生班级

表名: Course		说明: 课程信息表			
属性列	数据类型	长度	空值	列约束	说明
Cno	char()	3	Not Null	PK	课程编号
Cname	char()	12	Not Null		课程名称
Tno	integer		Null		教师编号
Chours	Float(1)		Null		课程学时
Credit	char()	3	Null		课程学分

表名: SC		说明: 选课表			
属性列	数据类型	长度	空值	列约束	说明
Sno	char()	8	Not Null	FK	学生学号
Cno	char()	3	Not Null	FK	课程编号
Score	float(1)		Null		课程成绩

表名: dept		说明: 院系信息表			
属性列	数据类型	长度	空值	列约束	说明
Dno	char()	2	Not Null		系编号
Dname	char()	10	Not Null		系名称
Dean	char()	10	Null		系主任

表名: Teacher		说明: 教师信息表			
属性列	数据类型	长度	空值	列约束	说明
Tno	char()	3	Not Null	FK	教师编号
Tname	char()	10	Not Null		教师姓名
Dno	char()	2	Null		所属系号
Salary	float(2)		Null		教师工资

查询student表中03系的学生学号、姓名、性别和所在系编号

SELECT Sno, Sname, Ssex, Dno FROM student WHERE Dno = '03';

查询课程号为001的所有学生学号和成绩，并从高到低进行排序

SELECT Sno, Score FROM SC where cno='001' ORDER BY Score desc;

查询选修了003号课程且成绩在60以下的学生学号

SELECT Sno FROM SC WHERE (Cno='003') AND (score<60);

按 ‘数据库’ 课成绩由低到高顺序显示所有同学姓名和成绩

Select Sname, Score From Student, SC, Course Where Student.Sno = SC.Sno and SC.Cno = Course.Cno and Cname = '数据库' Order By Score ASC;

求 “002” 号课成绩比 “003” 号课成绩高的所有学生的学号

Select S1.Sno From SC S1, SC S2 Where S1.Sno = S2.Sno and S1.Cno='002' and S2.Cno='003' and S1.Score > S2.Score;

目录

CONTENTS

5

SQL复杂查询

5.1 聚合函数

5.2 分组查询与过滤

5.3 嵌套查询

5.4 集合操作

5.1 聚合函数

- SQL提供了五个作用在简单列值集合上的内置聚集函数agfunc, 分别是: COUNT、SUM、AVG、MAX、MIN
- SQL聚集函数的参数类型、结果类型与作用如下:

Name	Argument type	Result type	Description
Count	any (can be *)	numeric	count of occurrences
sum	numeric	numeric	sum of arguments
avg	numeric	numeric	average of arguments
max	char or numeric	same as arg	maximum value
min	char or numeric	same as arg	minimum value

求个数

求和

求平均

求最大

求最小

5.1 聚合函数

聚合函数

示例：求教师的工资总额

```
Select Sum(Salary) From Teacher;
```

Teacher

T#	Tname	D#	Salary
001	赵三	01	1200.00
002	赵四	03	1400.00
003	赵五	03	1000.00
004	赵六	04	1100.00

Dept

D#	Dname	Dean
01	机电	李三
02	能源	李四
03	计算机	李五
04	自动控制	李六

SC

S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

示例：求计算机系教师的工资总额

```
Select Sum(Salary) From Teacher T, Dept Where
Dept.Dname = '计算机' and Dept.D# = T.D#;
```

示例：求数据库课程的平均成绩

```
Select AVG(Score) From Course C, SC Where
C.Cname = '数据库' and C.C# = SC.C#;
```

5.1 聚合函数

聚合函数

Null在聚合函数中的角色

- Count 为 0
- 其余均忽略 Null

Sum: 不将其计入

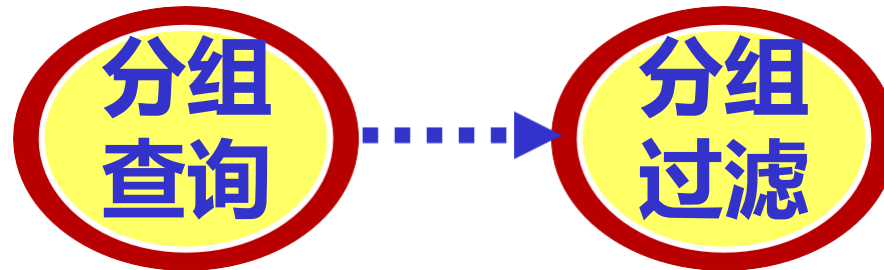
Avg: 具有 Null 的元组不参与

Max / Min: 不参与

怎么分组过滤?

下面的查询请求该如何表达呢?

- 求每一门课程的平均成绩
- 求每一个学生的平均成绩



怎么分组过滤?

Group By子句

分组: SQL可以将检索到的元组按照某一条件进行分类, 具有相同条件值的元组划到一个组或一个集合中, 同时处理多个组或集合的聚集运算。

➤ 分组的基本语法:

Select 列名 | **expr** | **agfunc**(列名) [[, 列名 | **expr** | **agfunc**(列名)] ...]

From 表名1 [, 表名2 ...]

[**Where** 检索条件]

[**Group by** 分组条件] ;

➤ 分组条件可以是

列名1, 列名2, ...

怎么分组过滤?

Group By子句

示例：求每一个学生的平均成绩

```
Select S#, AVG(Score) From SC
Group by S#;
```

按学号进行分组，即学号相同的元组划到一个组中并求平均值

S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

示例：求每一门课程的平均成绩

```
Select C#, AVG(Score) From SC
Group by C#;
```

按课号进行分组，即课号相同的元组划到一个组中并求平均值

S#	C#	Score
98030101	001	92
98040202	001	55
98030102	001	54
98030101	002	85
98030102	002	85
98040202	002	90
98040202	003	80
98030101	003	88
98040203	003	56
98030102	003	48

5.2 分组查询与过滤

- 聚集函数是不允许用于Where子句中的：Where子句是对每一元组进行条件过滤，而不是对集合进行条件过滤

分组过滤：若要对集合(即分组)进行条件过滤，即满足条件的集合/分组留下，不满足条件的集合/分组剔除。

- **Having子句，又称分组过滤子句。** 需要有Group by子句支持，换句话说，没有Group by子句，便不能有Having子句。

Select 列名 | **expr** | **agfunc(列名)** [[, 列名 | **expr** | **agfunc(列名)**] ...]

From 表名1 [, 表名2 ...]

[**Where** 检索条件]

[**Group by** 分组条件 [**Having** 分组过滤条件]] ;

5.2 分组查询与过滤

示例：求不及格课程超过两门的同学的学号

```
Select S# From SC
Where Score < 60
Group by S# Having Count(*)>2;
```

示例：求有10人以上不及格的课程号

```
Select C# From SC
Where Score < 60
Group by C# Having Count(*)>10;
```

SC

S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

5.2 分组查询与过滤

Group By子句

Having 与 Where的区别

- Where 决定哪些元组被选择参加运算，作用于关系中的元组
- Having 决定哪些分组符合要求，作用于分组
- 聚合函数的条件关系必须用Having，Where中不应出现聚集函数

每一分组检查满足与否的条件要用 **Having 子句** 表达。

注意:不是每一行 都检查，所以使用 Having子句一定要有Group by 子句

S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

每一行都要检查满足与否的条件要用 **WHERE子句** 表达

怎么嵌套查询?

➤现实中, 很多情况需要进行下述条件的判断

- ❑ 集合成员资格

某一元素是否是某一个集合的成员

- ❑ 集合之间的比较

某一个集合是否包含另一个集合等

- ❑ 集合基数的测试

测试集合是否为空

测试集合是否存在重复元组

三种类型的子查询:

(NOT) IN-子查询;

-Some/ -All子查询;

(NOT) EXISTS子查询

子查询:

出现在Where子句中的Select语句被称为子查询(subquery),子查询返回了一个集合, 可以通过与这个集合的比较来确定另一个查询集合。

三种类型的子查询: (NOT) IN-子查询; θ -Some/ θ -All子查询; (NOT) EXISTS子查询

5.3 嵌套查询

(NOT) IN子查询

➤基本语法:

表达式 [*not*] *in* (子查询)

- 语法中，表达式的最简单形式就是列名或常数。
- 语义：判断某一表达式的值是否在子查询的结果中。

示例：列出张三、王三同学的所有信息

```
Select * From Student  
Where Sname in (“张三”, “王三”);
```

//此处直接使用了某一子查询的结果集合。如果该集合是已知的固定的，可以如上直接书写

➤上述示例相当于

```
Select * From Student  
Where Sname = “张三” or Sname = “王三”;
```


5.3 嵌套查询

(NOT) IN子查询

Student					
S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

示例：列出选修了001号课程的学生的学号和姓名

```
Select S#, Sname From Student
Where S# in ( Select S# From SC Where C# = '001' );
```

示例：求既学过001号课程, 又学过002号课程的学生的学号

```
Select S# From SC
Where C# = '001' and
      S# in ( Select S# From SC Where C# = '002' );
```

5.3 嵌套查询

(NOT) IN子查询

Student					
S#	Sname	Ssex	Sage	D#	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

SC		
S#	C#	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56
98030102	001	54
98030102	002	85
98030102	003	48

示例：列出没学过李明老师讲授课程的所有同学的姓名？

```
Select Sname From Student
Where S# not in ( Select S# From SC, Course C, Teacher T
                  Where T.Tname = '李明' and SC.C# = C.C#
                  and T.T# = C.T# );
```

➤注意下述写法的不正确性！

```
Select Sname From Student S, SC, Course C, Teacher T
Where T.Tname <> '李明' and C.C# = SC.C#
      and SC.S# = S.S# and T.T# = C.T#;
```

5.3 嵌套查询

(NOT) IN子查询

- 带有子查询的Select语句区分为内层和外层

外层查询

```
Select Sname
```

```
From Student
```

```
Where S# not in (
```

内层查询

```
    Select S#
```

```
    From SC, Course C, Teacher T
```

```
    Where T.Tname = '李明' and SC.C# = C.C#
```

```
          and T.T# = C.T# );
```

- **非相关子查询**：内层查询独立进行，没有涉及任何外层查询相关信息的子查询

- 前面的子查询示例都是非相关子查询

5.3 嵌套查询

some/all子查询

θ some / θ all子查询

➤ 基本语法:

表达式 θ some (子查询)

表达式 θ all (子查询)

➤ 语法中, θ 是比较运算符: $<$, $>$, $>=$, $<=$, $=$, $<>$ 。

➤ 语义: 将表达式的值与子查询的结果进行比较:

□ 如果表达式的值至少与子查询结果的某一个值相比较满足 θ 关系, 则“表达式 θ some (子查询)”的结果便为真;

□ 如果表达式的值与子查询结果的所有值相比较都满足 θ 关系, 则“表达式 θ all (子查询)”的结果便为真;

5.3 嵌套查询

some/all子查询

示例：找出工资最低的教师姓名

```
Select  Tname  From Teacher  
Where  Salary <= all ( Select  Salary From Teacher );
```

示例：找出001号课成绩不是最高的所有学生的学号

```
Select  S#  From SC  
Where  C# = "001" and  
       Score < some ( Select  Score From SC Where  C# = "001" );
```

5.3 嵌套查询

some/all子查询

□ 找出001号课成绩最高的所有学生的学号

```
Select  S#  From  SC
Where  C# = "001" and
       Score >= all ( Select  Score From  SC Where  C# = "001" );
```

□ 找出98030101号同学成绩最低的课程号

```
Select  C#  From  SC
Where  S# = "98030101" and
       Score <= all ( Select  Score From  SC Where S# = "98030101" );
```

□ 找出张三同学成绩最低的课程号

```
Select  C#  From  SC, Student S
Where  Sname = "张三" and S.S#=SC.S# and
       Score <= all ( Select  Score From  SC
                      Where S#=S.S# );
```

5.3 嵌套查询

some/all子查询

等价性变换需要注意

➤ 如下两种表达方式含义是相同的

表达式 **= some** (子查询)

表达式 **in** (子查询)

示例

```
Select  Sname  From Student S
Where  S# in ( Select  S#  From SC
                Where  S# = S.S#  and C# = '001' );
```

```
Select  Sname  From Student S
Where  S# = some ( Select  S#  From SC
                Where  S# = S.S#  and C# = '001' );
```

5.3 嵌套查询

exists子查询 (空关系的测试)

➤ 基本语法:

[not] Exists (子查询)

➤ 语义: 子查询结果中有无元组存在

示例: 检索选修了赵三老师主讲课程的所有同学的姓名

```
Select DISTINCT Sname From Student
Where exists ( Select * From SC, Course, Teacher
               Where SC.C# = Course.C# and SC.S# = Student.S#
               and Course.T# = Teacher.T# and Tname = '赵三' );
```

➤ 不加not形式的Exists谓词可以不用, 比如上面例子就可以直接写成:

```
Select DISTINCT Sname From Student, SC, Course, Teacher
Where SC.C# = Course.C# and SC.S# = Student.S#
      and Course.T# = Teacher.T# and Tname = '赵三' );
```


5.3 嵌套查询

exists子查询

➤然而not Exists却可以实现很多新功能

示例：检索学过001号教师主讲的所有课程的所有同学的姓名

```
Select Sname From Student
```

```
Where not exists
```

//不存在

```
( Select * From Course
```

//有一门001教师主讲课程

```
Where Course.T# = '001' and not exists
```

//该同学没学过

```
( Select * From SC
```

```
Where S# = Student.S# and C# = Course.C# ) );
```

➤上述语句的意思：不存在有一门001号教师主讲的课程该同学没学过

■ 如何利用SQL语言实现关系代数操作?

SQL语言：并运算**UNION**，交运算**INTERSECT**，差运算**EXCEPT**。

➤基本语法形式：

子查询{ **Union [ALL]** | **Intersect [ALL]** | **Except [ALL]** 子查询 }

➤通常情况下自动删除重复元组：不带**ALL**。若要保留重复的元组，则要带**ALL**。

示例：求学过002号课的同学或学过003号课的同学学号

```
Select S# From SC Where C# = '002'  
UNION  
Select S# From SC Where C# = '003';
```

```
Select S# From SC Where C# = '002' OR C# = '003';
```

示例：求既学过002号课，又学过003号课的同学学号

```
Select S# From SC Where C# = '002'  
INTERSECT  
Select S# From SC Where C# = '003';
```

```
Select S# From SC Where C# = '002' and S# IN  
(Select S# From SC Where C# = '003');
```

假定所有学生都有选课，求没学过002号课程的学生学号

```
Select DISTINCT S# From SC  
EXCEPT  
Select S# From SC Where C# = '002';
```

```
Select DISTINCT S# From SC SC1  
Where not exists ( Select * From SC  
Where C# = '002' and S# = SC1.S#) ;
```

怎么内连接、外连接?

Select 列名[, 列名] ...]
From 表名1 [NATURAL
 [INNER | { LEFT | RIGHT | FULL } [OUTER]] JOIN 表名2
 { ON 连接条件 | Using (Colname {, Colname ...}) }
 [**Where** 检索条件] ... ;

连接类型(四者选一)	连接条件(三者选一)
inner join	natural
left outer join	on <连接条件>
right outer join	using (Col ₁ , Col ₂ , ..., Col _n)
full outer join	

求所有教师的任课情况

Select Teacher.T#, Tname, Cname
From Teacher Inner Join Course
ON Teacher.T# = Course.T#;

任课情况(内连接)

T#	Tname	Cname
001	赵三	数据库
001	赵三	编译原理
003	赵五	数据结构
003	赵五	C 语言
004	赵六	高等数学



Course				
C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
002	高等数学	80	12	004

Teacher			
T#	Tname	D#	Salary
001	赵三	01	1200.00
002	赵四	03	1400.00
003	赵五	03	1000.00
004	赵六	04	1100.00

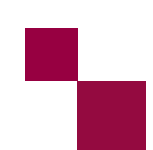


求所有教师的任课情况(没有任课的教师也需列在表中)

Select Teacher.T#, Tname, Cname
From Teacher Left Outer Join Course
ON Teacher.T# = Course.T#;

任课情况(左外连接)

T#	Tname	Cname
001	赵三	数据库
001	赵三	编译原理
002	赵四	
003	赵五	数据结构
003	赵五	C 语言
004	赵六	高等数学



SELECT语句是SQL的核心语句,其语句成份多样,尤其是目标类表达式和条件表达式,可以有多种可选形式.这里总结一下它们的一般格式.

SELECT语句的一般格式:

```
SELECT[ALL|DISTINCT]<目标类表达式>[别名]...  
FROM<表名或视图名>[别名][,<表名或视图名>[别名]].....  
[WHERE<条件表达式>]  
[GROUP BY <列名1>[HAVING<条件表达式>]]  
[ORDER BY <别名2>[ASC|DESC]];
```

目录

CONTENTS

6

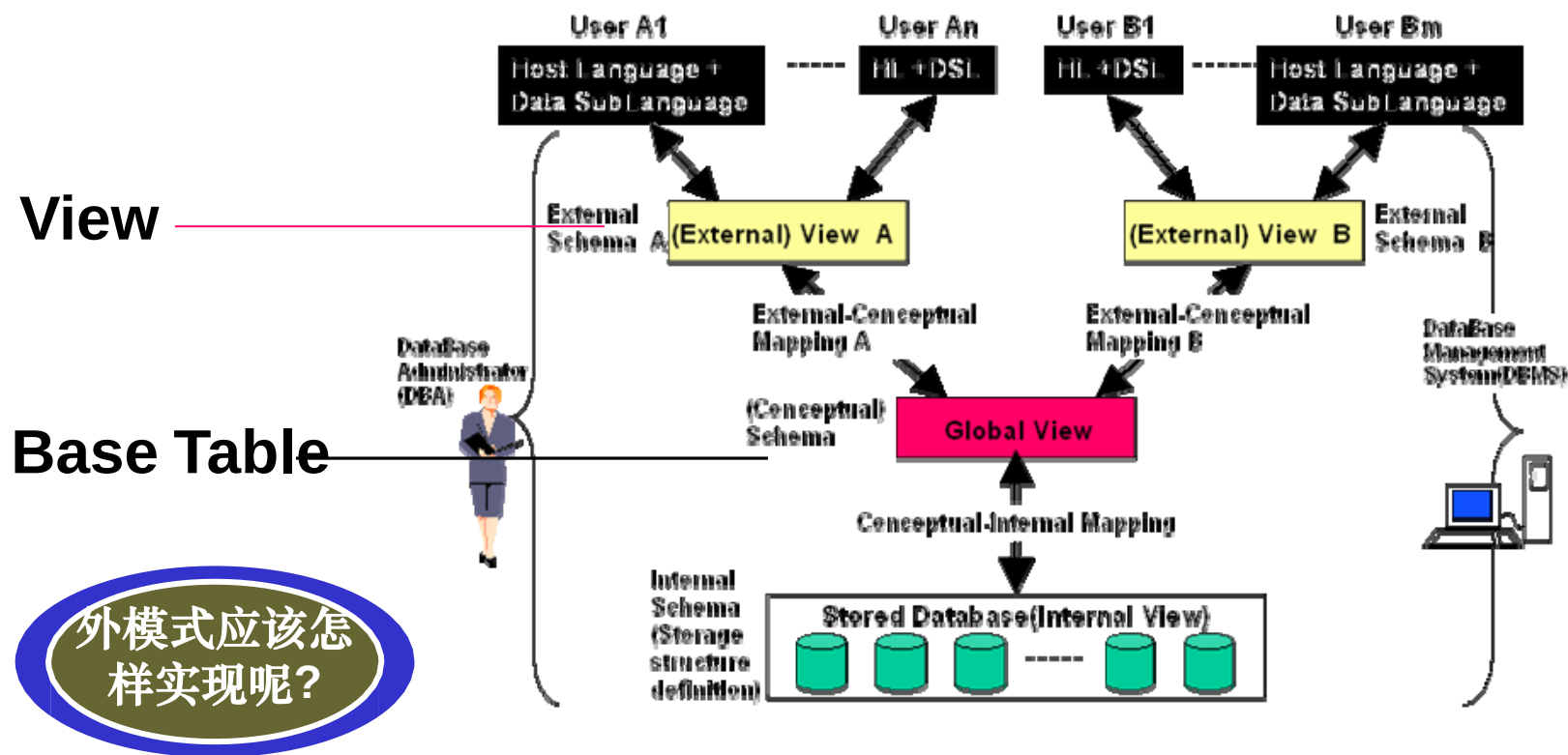
视图

- 6.1 视图概述
- 6.2 建立视图
- 6.3 查询视图
- 6.4 更新视图
- 6.5 删除视图
- 6.6 视图的作用

6.1 视图概述

回顾：三级模式两层映像结构

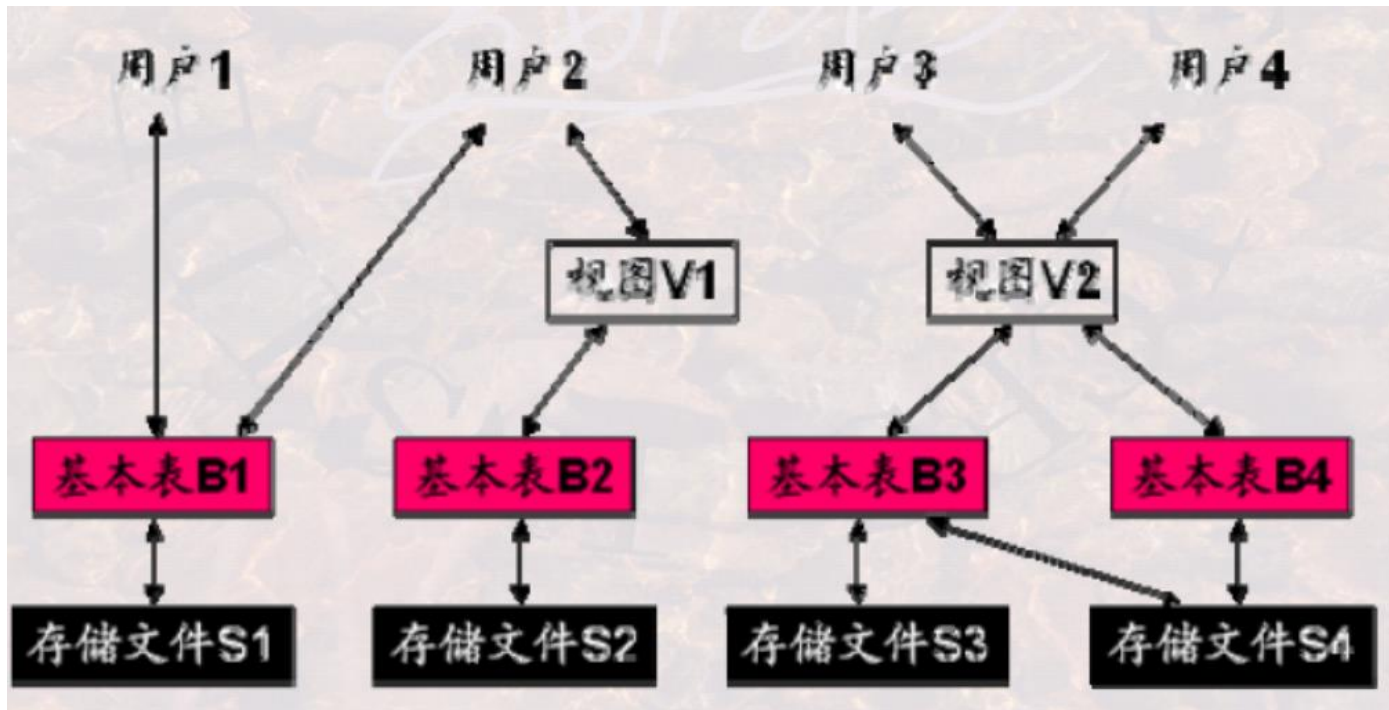
➤ 对应概念模式的数据在SQL中被称为**基本表(Table)**, 而对应外模式的数据称为**视图(View)**。视图不仅包含外模式, 而且包含其E-C映像。



6.1 视图概述

视图概念

- 虚表，是从一个或几个基本表（或视图）导出的虚拟表
- 只存放视图的定义，不会出现数据冗余
- 基表中的数据发生变化，从视图中查询出的数据也随之改变



6.2 建立视图

建立视图

- 视图需要 “先定义，再使用”

定义视图

```
create view view_name [(列名[, 列名] ...)]  
as 子查询 [with check option]
```

- 如果视图的属性名缺省，则默认为子查询结果中的属性名；也可以显式指明其所拥有的列名。
- with check option 指明当对视图进行 insert, update, delete 时，要检查进行 insert/update/delete 的元组是否满足视图定义中子查询中定义的条件表达式

6.2 建立视图

建立视图

示例：定义一个视图 CompStud 为计算机系的学生，通过该视图可以将Student表中其他系的学生屏蔽掉

```
Create View CompStud AS  
    ( Select * From Student  
      Where D# in ( Select D# From Dept  
                    Where Dname = '计算机' ) );
```

示例：定义一个视图Teach为教师任课的情况，把Teacher表中的个人隐私方面的信息，如工资等屏蔽掉，仅反映其教哪门课及其学分等。

```
Create View Teach AS  
    ( Select T.Tname, C.Cname, Credit  
      From Teacher T, Course C  
      Where T.T# = C.T# );
```

6.2 建立视图

建立分组视图

使用视图：定义好的视图，可以像Table一样，在SQL各种语句中使用

示例：检索主讲数据库课程的教师姓名，我们可使用Teach

```
Select T.Tname From Teach T  
Where T.Cname = '数据库' ;
```

示例：检索计算机系的所有学生，我们可使用CompStud

```
Select * From CompStud;
```

示例：检索计算机系的年龄小于20的所有学生，我们可使用CompStud

```
Select * From CompStud  
Where Sage < 20 ;
```

```
Create View Teach AS  
( Select T.Tname, C.Cname, Credit  
  From Teacher T, Course C  
  Where T.T# = C.T# );
```

```
Create View CompStud AS  
( Select * From Student  
  Where D# in ( Select D# From Dept  
                Where Dname = '计算机' );
```

6.3 查询视图

查询视图

- 定义视图，有时可方便用户进行检索操作。

示例：定义视图StudStat, 描述学生的平均成绩、最高成绩，最低成绩等

```
Create View StudStat(S#, Sname, AvgS, MinS, MaxS, CNT)
as ( Select S#, Sname, AVG(Score), MIN(Score), Max(Score), Count(*)
      From Student S, SC Where S.S# = SC.S#
      Group by S.S# );
```

示例：基于视图StudStat检索某一学生平均成绩

```
Select Sname, AvgS From StudStat Where Sname = '张三';
```

6.4 查询视图

更新视图

SQL视图更新：是比较复杂的问题，因视图不保存数据，对视图的更新最终要反映到对基本表的更新上，而有时，视图定义的映射不是可逆的。

示例：

```
create view S_G(S#, Savg )  
as ( select S#, AVG(Score)  
      from SC group by S# );
```

如要进行下述更新操作？

```
update S_G  
set Savg = 85  
where S# = '98030101';
```

能否由视图S_G的更新，而更新SC呢？

哪些类型的视图是
不可更新的？

哪些类型的视图又
是可更新的？

6.4 更新视图

更新视图

示例

```
create view ClassStud(Sname, Sclass)
as ( select Sname, Sclass
      from Student );
```

➤ 如要进行下述更新操作？

```
Insert into ClassStud
Values ( '张三', '980301' );
```

➤ 能否由视图ClassStud的更新，而更新Student呢？

➤ 回答是：不能，因为缺少S#，而S#是Student的主键。

6.4 更新视图

更新视图

战德臣 教授

SQL视图更新的可执行性

- ❑ 如果视图的select目标列包含聚集函数，则不能更新
- ❑ 如果视图的select子句使用了unique或distinct，则不能更新
- ❑ 如果视图中包括了group by子句，则不能更新
- ❑ 如果视图中包括经算术表达式计算出来的列，则不能更新
- ❑ 如果视图是由单个表的列构成，但并没有包括主键，则不能更新
- 对于由单一Table子集构成的视图，即如果视图是从单个基本表使用选择、投影操作导出的，并且包含了基本表的主键，则可以更新

6.4 更新视图

更新视图

可更新SQL视图示例：

```
create view CStud(S#, Sname, Sclass)
as ( select S#, Sname, Sclass
      from Student where D#='03');
```

➤ 上例是可以更新的

```
Insert into CStud
Values ( “98030104”, “张三丰”, “980301” );
```

```
insert into CStud
values (“98030104”, “张三丰”, “980301” )
```

↓ 转换为

```
insert into Student
values (“98030104”, “张三丰”, Null, Null, “03” , “980301” )
```

6.5 撤销视图

- 已经定义的视图也可以撤销

撤销视图

Drop View view_name

示例：撤销视图Teach

Drop View Teach;

示例：撤销视图CompStud

Drop View CompStud;

6.6 视图的作用

视图的作用

1. 视图能够简化用户的操作
2. 视图使用户能以多种角度看待同一数据（定制数据）
3. 视图对重构数据库提供了一定程度的逻辑独立性
4. 视图能够对机密数据提供安全保护

6.6 视图的作用

视图对重构数据库提供了一定程度的逻辑独立性

例：数据库逻辑结构发生改变

学生关系Student(Sno, Sname, Ssex, Sage, Sdept)“垂直”地分成两个基本表：

SX(Sno, Sname, Sage)

SY(Sno, Ssex, Sdept)

通过建立一个视图Student：

```
CREATE VIEW Student(Sno, Sname, Ssex, Sage, Sdept)
AS
SELECT SX.Sno, SX.Sname, SY.Ssex, SX.Sage, SY.Sdept
FROM SX, SY
WHERE SX.Sno=SY.Sno;
```

使用户的外模式保持不变，从而对原Student表的查询程序不必修改

6.6 视图的作用

视图对重构数据库提供了一定程度的逻辑独立性

视图可以使应用程序和数据库表在一定程度上独立。如果没有视图，应用一定是建立在表上的。有了视图之后，程序可以建立在视图之上，从而程序与数据库表被视图分割开来。视图可以在以下几个方面使程序与数据独立：

- 1 如果应用建立在数据库表上，当数据库表发生变化时，可以在表上建立视图，通过视图屏蔽表的变化，从而应用程序可以不动。
- 2 如果应用建立在数据库表上，当应用发生变化时，可以在表上建立视图，通过视图屏蔽应用的变化，从而使数据库表不动。
- 3 如果应用建立在视图上，当数据库表发生变化时，可以在表上修改视图，通过视图屏蔽表的变化，从而应用程序可以不动。
- 4 如果应用建立在视图上，当应用发生变化时，可以在表上修改视图，通过视图屏蔽应用的变化，从而数据库可以不动

6.6 视图的作用

视图能够对机密数据提供安全保护

视图可以作为一种安全机制。通过视图用户只能查看和修改他们所能看到的数据。其它数据库或表既不可见也不可以访问。如果某一用户想要访问视图的结果集，必须授予其访问权限。视图所引用表的访问权限与视图权限的设置互不影响。

视图的安全性可以防止未授权用户查看特定的行或列，使用户只能看到表中特定行的方法如下：

- 1 在表中增加一个标志用户名的列；
- 2 建立视图，是用户只能看到标有自己用户名的行；
- 3 把视图授权给其他用户。

目录

CONTENTS

7

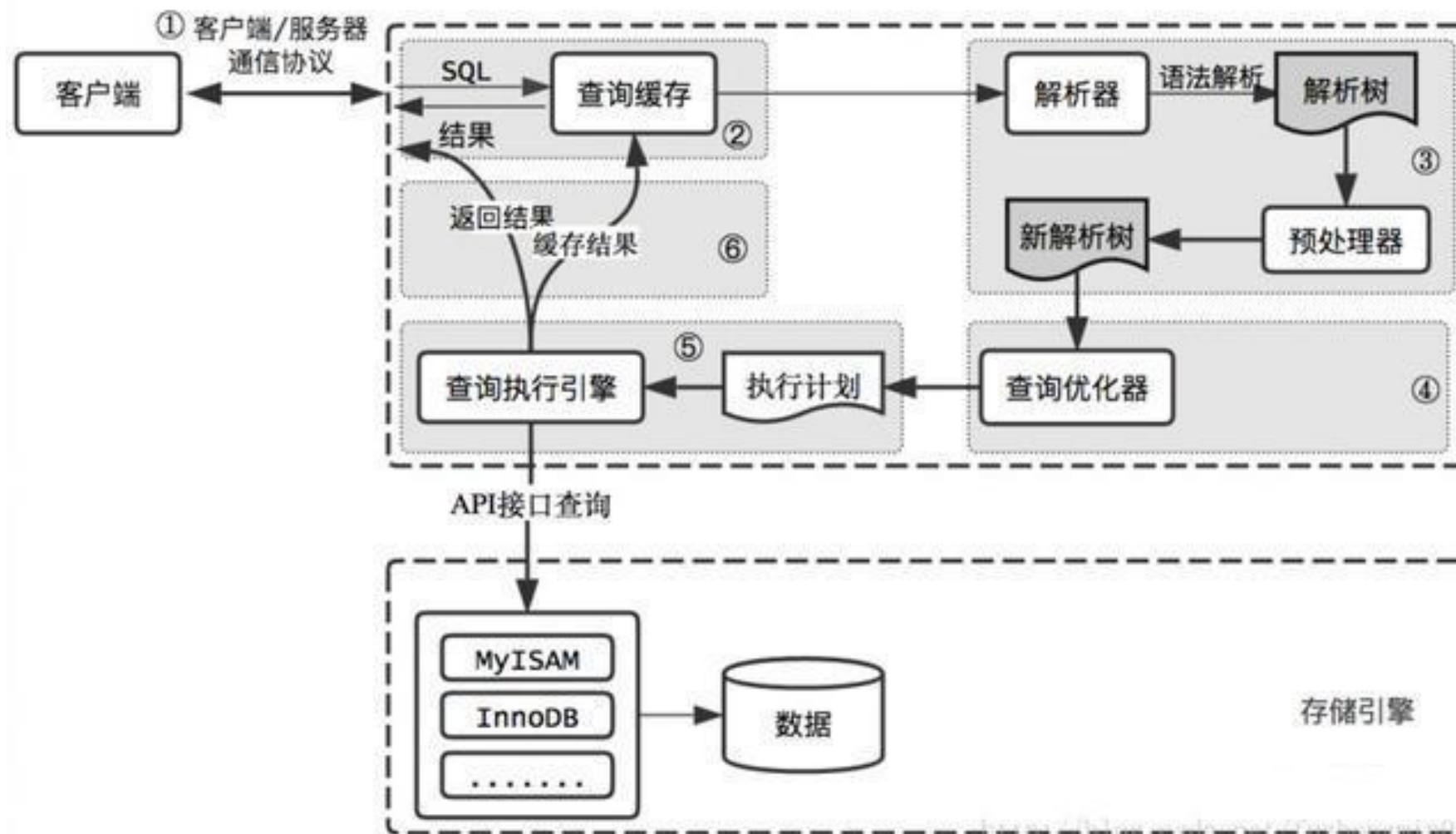
SQL查询是怎么实现和优化的?

7.1 SQL查询执行过程

7.2 SQL查询优化过程

7.1 SQL查询执行过程

一条SQL语句的执行流程



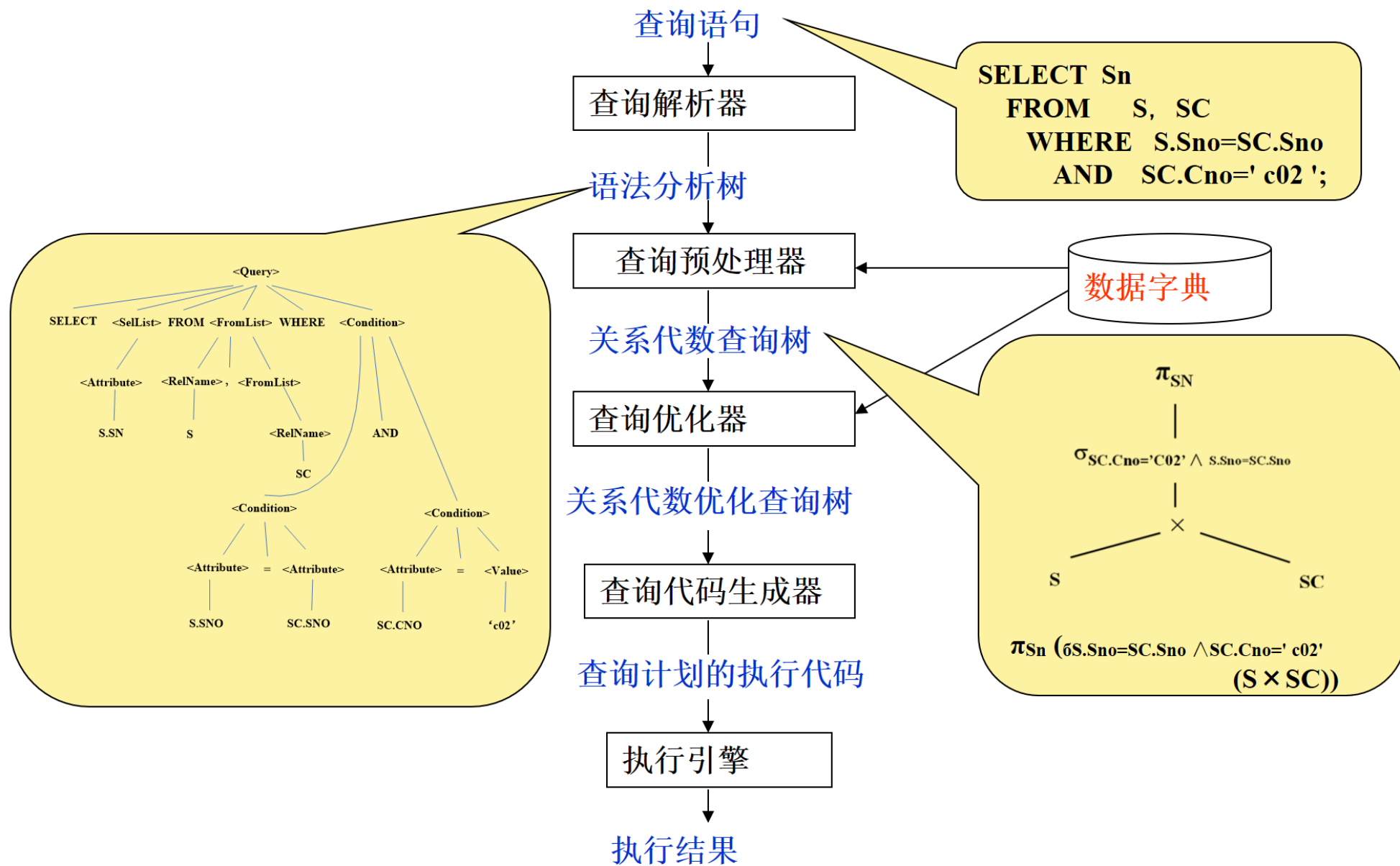
7.1 SQL查询执行过程

SQL语句的执行顺序

- (8)SELECT (9)DISTINCT (11)<Top Num> <column>
(1)FROM [left_table]
(3)<join_type> JOIN <right_table>
(2) ON <join_condition>
(4)WHERE <where_condition>
(5)GROUP BY <group_by_list>
(6)WITH <CUBE | RollUP>
(7)HAVING <having_condition>
(10)ORDER BY <order_by_list>

<https://blog.csdn.net/dxm809>

7.1 SQL查询执行过程



7.2 SQL查询优化过程

为什么需要查询优化

关系代数操作执行次序对效率的影响

➢ 如下三条关系代数语句表示同样的检索需求，但哪一个更好呢？

关系代数操作次序是否很重要呢？

Query 1: $\pi_{S\#, Sname, Score}(\sigma_{Cname="DB"}((Student \bowtie SC) \bowtie Course))$

10000	500000	
	500000	1000
		500000

Query 2: $\pi_{S\#, Sname, Score}(Student \bowtie (SC \bowtie (\sigma_{Cname="DB"}(Course))))$

		1000
	500000	1
10000	2500(约)	
2500(约)		

Query 3: $\pi_{S\#, Sname, Score}(\pi_{S\#, Sname}(Student) \bowtie (SC \bowtie (\pi_{C\#}(\sigma_{Cname="DB"}(Course)))))$

7.2 SQL查询优化过程

查询优化

“如何使数据库查询的执行时间最短？”

三个层面进行优化：

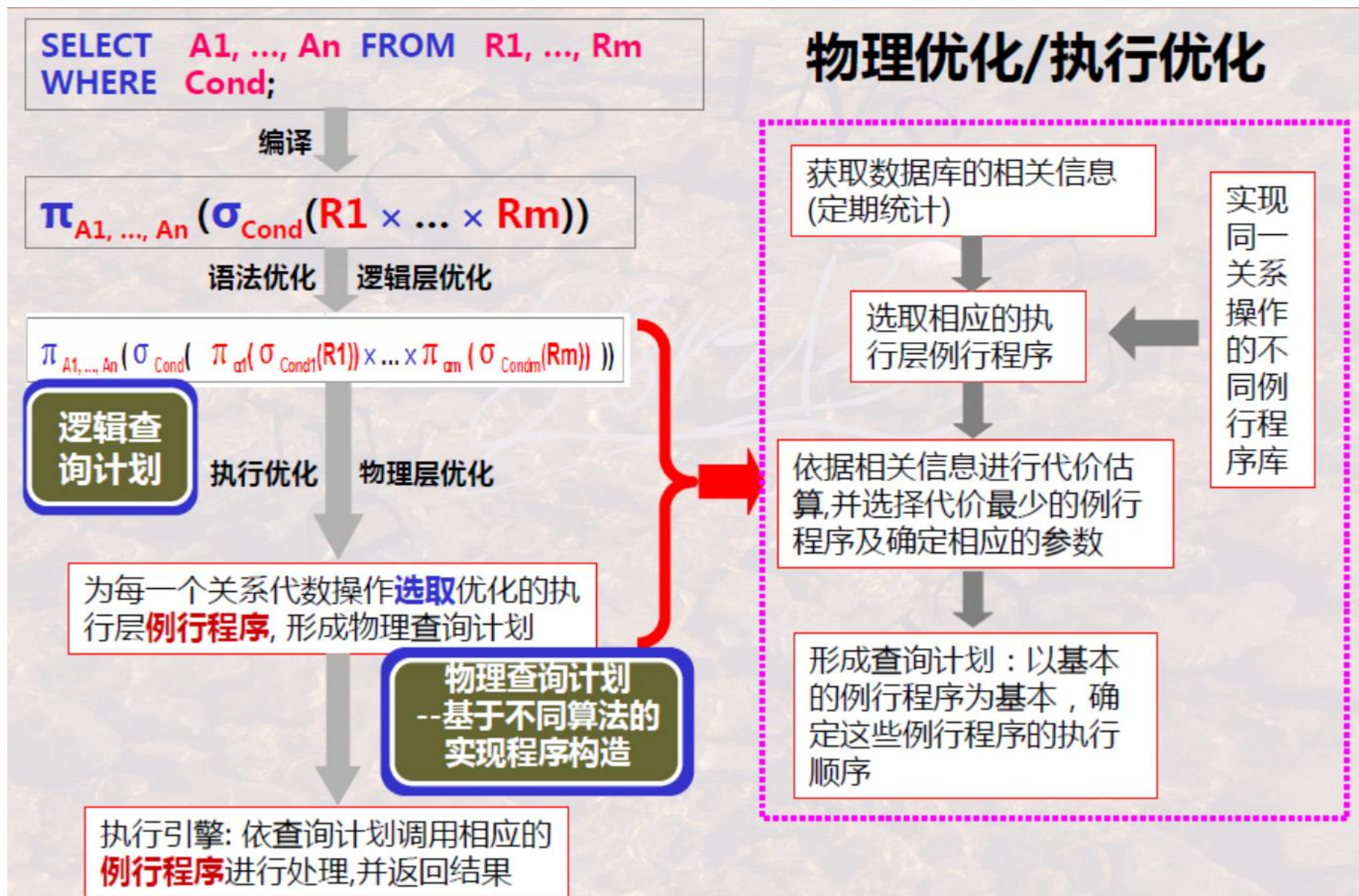
- 语义优化**：利用模型的语义及完整性规则，优化查询。
- 语法优化**---**逻辑层优化**：利用语法结构，优化操作执行顺序；
- 执行优化**---**物理层优化**：存取路径和执行算法的选择与执行次序优化；

7.2 SQL查询优化过程

查询优化



7.2 SQL查询优化过程



谢谢大家!

